Introduction to Docker Swarm & Docker Stack

---

1. Overview

Docker Compose simplified running multi-container applications on a single host. Docker Swarm takes this further by turning that single-host definition into a production-grade, distributed system spanning many machines. Swarm mode, built directly into the Docker Engine, provides native clustering: it groups multiple Docker hosts into a fault-tolerant cluster and lets you define services that can be replicated, scaled, updated with zero downtime, and exposed through an automatic load-balancing mesh. Docker Stack extends the Compose file format with swarm-specific options (`deploy` sections) so you can use the same familiar YAML to describe a production deployment. In this session you will initialise a single-node Swarm, write a stack file for the MERN application, and deploy and scale it with `docker stack deploy`.

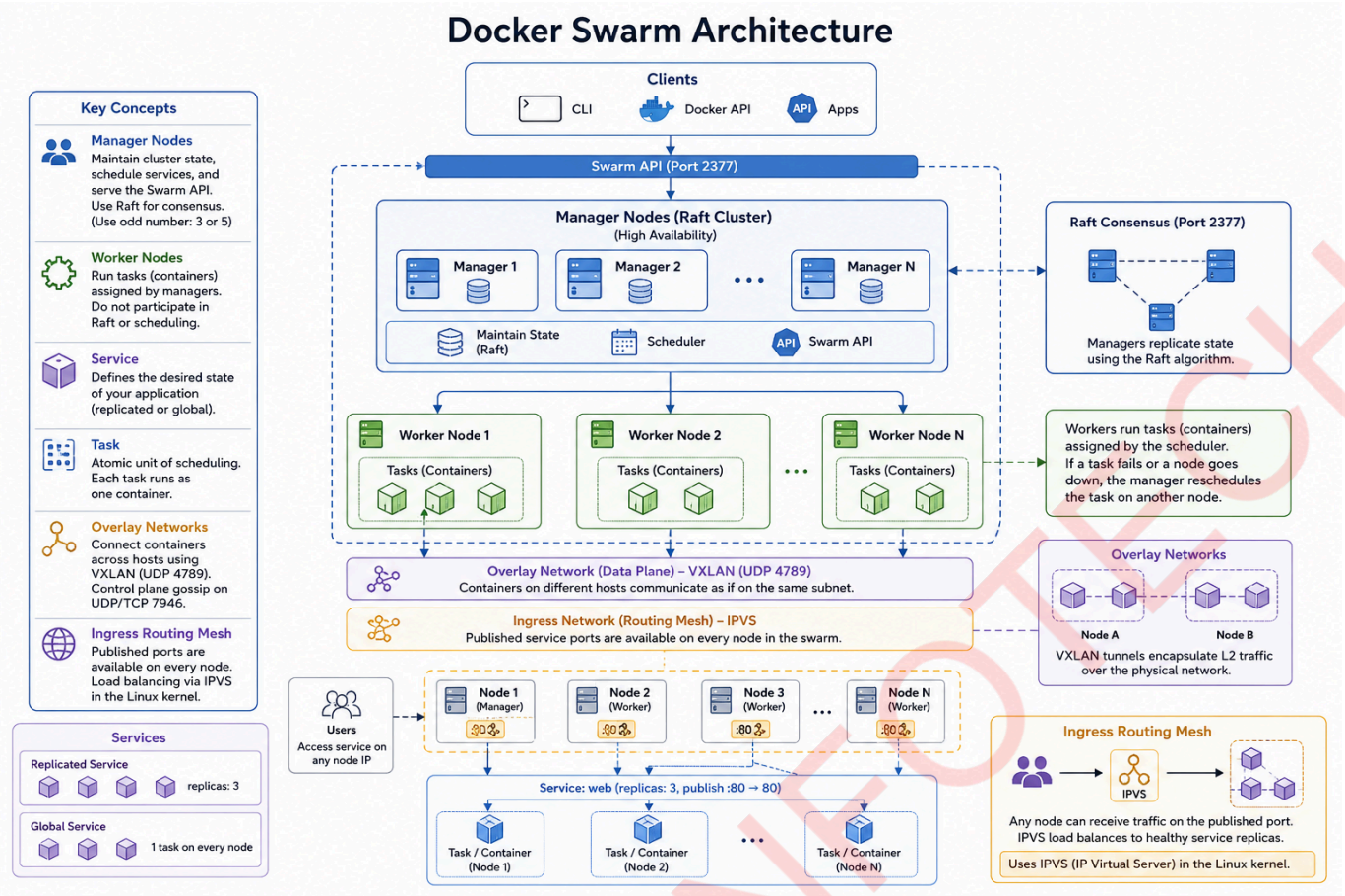
---

2. Learning Objectives

- Explain Docker Swarm mode architecture: nodes (manager and worker), services, tasks, and overlay networks.
- Describe the ingress routing mesh and how it makes any published port accessible on every node regardless of where the actual container is running.
- Understand the difference between `docker compose` (development, single host) and `docker stack` (production, multi-host, declarative desired-state).
- Adapt a Compose file for Swarm by adding `deploy` keys (replicas, update_config, resources, restart_policy).
- Distinguish between overlay and bridge networks in Swarm mode and create overlay networks for cross-node service communication.
- Initialise a Swarm cluster, view node information, and deploy a stack using `docker stack deploy`.
- Scale a service, perform a rolling update, and monitor tasks with `docker service ps`.
- Tear down a stack and leave the Swarm.

---

3. Docker Swarm Architecture



Swarm mode is activated by a single command (`docker swarm init`). Once initialised, the Docker Engine becomes a manager node and can accept additional hosts as managers or workers.

3.1 Nodes

- **Manager nodes:** Maintain the cluster state, schedule services, and serve the Swarm API. Multiple managers provide high availability via the Raft consensus algorithm (an odd number of managers is recommended, typically three or five).
- **Worker nodes:** Execute the containers assigned to them. Workers do not participate in Raft or scheduling decisions.

3.2 Services A service defines the desired state of an application component: which image, how many replicas, which ports to publish, how updates should be performed, and so on. Services can be:

- **Replicated:** Run a specified number of identical task copies across the cluster.
- **Global:** Run exactly one task on every eligible node (e.g., monitoring agents).

3.3 Tasks A task is the atomic unit of scheduling. When you define a service with `replicas: 3`, the manager creates three tasks. The scheduler assigns each task to a suitable node, where the task becomes a running container. If a container fails or a node goes down, the manager replaces the failed task on another node to maintain the desired replica count.

3.4 Overlay Networks Swarm uses overlay networks to connect containers across different hosts. Overlay networks use VXLAN tunnels (UDP port 4789) to encapsulate layer-2 traffic over the physical network, allowing containers on separate nodes to communicate as if they were on the same subnet. The control plane uses gossip protocols (UDP/TCP 7946) to distribute network state.

When a service is attached to an overlay network, every replica gets a unique, service-discoverable DNS name in the form `<service-name>` or `<service-name>.<stack-name>`. Swarm's internal DNS distributes requests across all containers backing a service (round-robin by default).

3.5 Ingress Routing Mesh When a service publishes a port (`--publish published=80,target=80`), that port is exposed on **every node** in the swarm, regardless of whether the node is running a replica of that service. Incoming traffic is routed to any healthy replica via the ingress network overlay. This means you can hit any node's IP on the published port and reach the service. The load-balancing uses IPVS (IP Virtual Server) in the Linux kernel, configured automatically by Swarm.

---

4. Docker Compose vs Docker Stack

Aspect	`docker compose`	`docker stack`
Scope	Single host	Swarm cluster (multiple hosts)
Orchestration	None – containers are started/stopped directly by Compose	Desired-state management by Swarm manager
File format	Compose Specification (any key allowed)	Compose Specification plus `deploy` section
`depends_on`	Supported, including `condition: service_healthy`	**Not supported** for stacks (ignored silently)
`build`	Supported – builds images on the target host	Not supported – must use images already built and pushed to a registry (or local images with `--resolve-image never`)
Networks	Default bridge driver	Default overlay driver when deployed as a stack
Secrets & Configs	Rudimentary support via file bind-mounts	Native Docker secrets and configs (encrypted in transit, at rest on managers)
Update strategy	Manual (`docker compose up --force-recreate`)	Rolling updates, blue-green via `update_config`

`docker stack` is the recommended way to run Compose files in production on a Swarm cluster. For local development, `docker compose` remains ideal.

---

5. The `deploy` Section in the Stack File

A Compose file intended for `docker stack deploy` must include `deploy:` under each service to control Swarm-specific behaviour. Key options:

```
services:
  my-service:
    image: my-image:tag
```

```
    deploy:
      replicas: 3
      placement:
        constraints:
          - "node.role==worker"
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
      restart_policy:
        condition: on-failure
      resources:
        limits:
          cpus: "2"
          memory: 512M
        reservations:
          cpus: "0.5"
          memory: 256M
```

- **replicas:** number of identical instances.
- **placement.constraints:** restrict scheduling using node attributes (e.g., `node.role==worker`, `node.labels.region==us-east`).
- **update_config:** controls rolling updates; `parallelism` is how many tasks update simultaneously, `delay` is wait time between batches, `order` can be `start-first` (new task started before old stopped, needs extra capacity) or `stop-first` (old stopped first).
- **restart_policy:** `condition` can be `none`, `on-failure`, or `any`; optional `delay`, `max_attempts`, `window`.
- **resources:** similar to Compose, but enforced by Swarm scheduler.

---

6. Demonstration

6.1 Prerequisites The project directory is `~/mern-app` with backend and frontend Dockerfiles and the code.

Update `backend/server.js` to add a connection retry:

```
// ... existing requires ...

async function initDB() {
    const poolConfig = {
        host: process.env.DB_HOST || "mysql",
        user: process.env.DB_USER || "mern_user",
        password: process.env.DB_PASSWORD || "mern_userpass",
        database: process.env.DB_NAME || "mern_app",
        waitForConnections: true,
        connectionLimit: 10,
    };
    // Retry up to 10 times, 5 seconds apart
    for (let i = 0; i < 10; i++) {
        try {
            db = await mysql.createPool(poolConfig);
```

```
            // Test the connection
            await db.query("SELECT 1");
            console.log("Database connected successfully");
            return;
        } catch (err) {
            console.log(`Database not ready (attempt ${i + 1}/10):
${err.message}`);
            await new Promise((resolve) => setTimeout(resolve, 5000));
        }
    }
    throw new Error("Could not connect to database after multiple attempts");
}
```

This ensures the backend will keep trying to reach MySQL during startup without requiring a health-check dependency.

6.2 Build and Push Images If you haven't already pushed images to Docker Hub, do so now. For simplicity we will use locally-built images because `docker stack deploy` can use them if the image is present on all nodes. In a single-node swarm, local images work.

```
cd ~/mern-app
docker compose build   # rebuild both images (or manually tag as before)
```

Verify images exist:

```
docker images | grep mern
```

To avoid tag confusion, ensure the images are tagged with a clear version:

```
docker tag mern-backend:latest mern-backend:swarm
docker tag mern-frontend:latest mern-frontend:swarm
```

6.3 Initialise Swarm

```
docker swarm init
```

Output:

```
Swarm initialized: current node (abc123...) is now a manager.

To add a worker to this swarm, run the following command:
    docker swarm join --token SWMTKN-1-... 192.168.1.10:2377
```

The current host becomes the single manager. Use `docker node ls` to verify:

```
docker node ls
# ID             HOSTNAME   STATUS  AVAILABILITY  MANAGER STATUS
# abc123... *    ubuntu     Ready   Active        Leader
```

Confirm Docker Swarm mode.

```
docker system info | grep "Swarm"
```

Output:

```
 Swarm: active
```

6.4 Create Stack File for MERN Create `stack.yaml` in the project root. We can reuse the same service definitions (in docker compose.yaml) but remove `depends_on` and add `deploy` sections.

```
# stack.yaml
version: "3.9" # kept for compatibility with older tooling; can be omitted on
Docker 29+

services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: devrootpass
      MYSQL_DATABASE: mern_app
      MYSQL_USER: mern_user
      MYSQL_PASSWORD: mern_userpass
    volumes:
      - mysqldata:/var/lib/mysql
    networks:
      - backend-net
    deploy:
      replicas: 1
      placement:
        constraints: [node.role == manager] # stick to a specific node for
persistent data
      restart_policy:
```

```
        condition: on-failure
    healthcheck:
      test:
        [
          "CMD",
          "mysqladmin",
          "ping",
          "-h",
          "localhost",
          "-u",
          "root",
          "-pdevrootpass",
        ]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 40s

  backend:
    image: mern-backend:swarm
    ports:
      - "5000:5000"
    environment:
      NODE_ENV: production
      DB_HOST: mysql
      DB_USER: mern_user
      DB_PASSWORD: mern_userpass
      DB_NAME: mern_app
    networks:
      - backend-net
      - frontend-net
    deploy:
      replicas: 2
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
      restart_policy:
        condition: any
    healthcheck:
      test:
        [
          "CMD-SHELL",
          'node -e "require(''http'').get(''http://localhost:5000/health'',
(r)=>process.exit(r.statusCode===200?0:1))"',
        ]
      interval: 20s
      timeout: 5s
      retries: 3

  frontend:
    image: mern-frontend:swarm
    ports:
      - "80:80"
```

```
    networks:
      - frontend-net
    deploy:
      replicas: 2
      update_config:
        parallelism: 1
        delay: 5s
        order: start-first
      restart_policy:
        condition: any

volumes:
  mysqldata:
    # Named volume; in a multi-node swarm you'd use a shared storage driver like
NFS
    driver: local

networks:
  frontend-net:
    driver: overlay
  backend-net:
    driver: overlay
```

Key Swarm adjustments:

- `version: "3.9"` is included for compatibility; modern Docker ignores it but it prevents warnings in older stack parsers. You can omit it on 29+.
- `depends_on` removed; `backend` uses retry logic to wait for MySQL.
- `deploy` sections define replicas, update strategy, and restart policy.
- Networks explicitly use `driver: overlay`. Without this, `docker stack deploy` defaults to overlay, but being explicit is good practice.
- Health checks are defined; Swarm uses them to determine task health and restart unhealthy containers.
- The MySQL service uses a placement constraint to keep it on the manager node, ensuring the data volume stays local. In a true multi-node setup you would use shared storage or a managed database service.

6.5 Deploy the Stack

```
docker stack deploy -c stack.yaml mern
```

The `mern` prefix is the stack name. Output:

```
Creating network mern_frontend-net
Creating network mern_backend-net
Creating service mern_mysql
Creating service mern_backend
Creating service mern_frontend
```

Wait a minute for MySQL to become healthy and the backend to start.

6.6 Check Services and Tasks

```
docker service ls
```

Shows services with current replica counts:

```
ID             NAME             MODE         REPLICAS   IMAGE
...            mern_mysql       replicated   1/1        mysql:8.4
...            mern_backend     replicated   2/2        mern-backend:swarm
...            mern_frontend    replicated   2/2        mern-frontend:swarm
```

To see individual tasks and which node each is on:

```
docker service ps mern_backend
```

In a single-node swarm all tasks run on the manager; the NODE column shows the hostname.

6.7 Verify Application Curl the backend via the published port (ingress mesh ensures it works even if hitting a node without a backend replica):

```
curl http://localhost:5000/
curl http://localhost:5000/health
curl http://localhost:5000/db-check
```

The frontend on port 80:

```
curl -I http://localhost/
```

Open a browser to your host's IP to see the React app.

6.8 Scale a Service

```
docker service scale mern_backend=4
```

Observe:

```
docker service ps mern_backend
```

New tasks appear and start; old ones remain until the desired count is reached. No downtime occurs because the frontend load-balances across all healthy backend replicas via the overlay network DNS.

6.9 Perform a Rolling Update Make a small change to the backend (e.g., update a message in `server.js`), rebuild the image, and retag it as `mern-backend:swarm`. Then update the service:

```
docker build -t mern-backend:swarm -f backend/Dockerfile .
docker service update --image mern-backend:swarm mern_backend
```

Or re-deploy the whole stack:

```
docker stack deploy -c stack.yaml mern
```

Because the `deploy.update_config` is set, tasks are updated one by one with a 10-second delay. Watch the update:

```
watch docker service ps mern_backend
```

Ctrl+C to exit.

6.10 Inspect the Ingress Mesh Publish a port and see that it's bound on all interfaces:

```
sudo ss -tlnp | grep :5000
# Shows listing on *:5000 by a process
docker network inspect ingress
```

The ingress network is a special overlay network used by all published services.

6.11 Tear Down

```
docker stack rm mern
```

This removes services, networks, and the stack record. Containers are stopped and removed. Named volumes are **not** deleted:

```
docker volume ls
# mysqldata volume still exists
```

To remove the volume as well: `docker volume rm mern_mysqldata` (the volume name is prefixed with the stack name).

Leave Swarm mode (if no longer needed):

```
docker swarm leave --force
```

This wipes the Swarm state from this node.

---

7. Important Considerations

- **Stateful services:** Running databases in Swarm is possible but requires careful handling of persistent storage. In production, managed databases outside the cluster or replicated storage solutions are common.
- **Image availability:** In a multi-node swarm, images must either be present on all nodes or pulled from a registry. Use a private registry or Docker Hub.
- **Secrets and configs:** Swarm supports encrypted Docker secrets (e.g., database passwords) and configs (configuration files) that are injected into containers without being baked into images. These were not used in this lab for brevity but are essential for production.
- `docker stack deploy` **is idempotent:** running it again with changes updates the services in a rolling fashion.
- **Monitoring:** `docker service logs` is not built-in (as of 2026, experimental only); rely on `docker service ps` and external logging drivers (e.g., Fluentd, ELK) for production logs.

---

8. Summary

- Docker Swarm mode is a built-in orchestrator that creates a cluster of Docker engines, managed as a single system. It provides service discovery, load-balancing, scaling, rolling updates, and self-healing.
- Nodes are managers (control plane) or workers (execute tasks). A service is a desired-state declaration; tasks are the actual container instances.
- Overlay networks use VXLAN tunnels to connect containers across nodes. The ingress mesh routes published ports to healthy replicas regardless of node.
- `docker stack deploy` reads a Compose file extended with `deploy` sections and manages the full lifecycle of the application in a Swarm. It supplements `docker compose` for production use.
